

Especificación de la Aplicación de Gestión de una Escuela de Yoga

Introducción

Este documento describe en detalle las especificaciones para el desarrollo de una aplicación de gestión integral de una escuela de yoga. El objetivo es **automatizar y simplificar las tareas administrativas** de la escuela, que actualmente podrían estar realizándose de forma manual (por ejemplo, mediante papel o hojas de cálculo básicas). La profesora Minouche será la **única usuaria** del sistema, utilizándolo en un portátil con recursos limitados. Por ello, se enfatiza la **simplicidad, ligereza y usabilidad** en el diseño de la aplicación.

La aplicación abarcará los principales aspectos de la gestión de un centro de yoga, incluyendo la administración de datos de alumnos, control de pagos de sus cuotas, gestión de la facturación e ingresos, registro de gastos (distinguiendo entre gastos fijos y variables) y generación de informes. Asimismo, se contemplan requisitos técnicos como el entorno operativo (Windows) y consideraciones de implementación (uso de tecnologías con **mínimos requisitos de instalación**).

Objetivos del Sistema

- **Centralizar la gestión de la escuela:** reunir en una sola aplicación toda la información relevante (alumnos, pagos, gastos, etc.) para tener **visibilidad completa** del funcionamiento del centro de yoga.
- **Facilitar el seguimiento de alumnos y sus pagos:** permitir a la profesora llevar un control claro de qué alumnos están al día en sus cuotas mensuales y cuáles tienen pagos pendientes, evitando confusiones o atrasos.
- **Llevar la contabilidad básica del centro:** registrar ingresos (cuotas de alumnos) y gastos (costos del negocio) para obtener un **balance simple** y conocer qué se está gastando y en qué. Esto ayuda a mantener la salud financiera del estudio.
- **Ahorrar tiempo y reducir errores:** automatizar la generación de **recibos, facturas, listados e informes**, eliminando cálculos manuales y reduciendo riesgos de errores humanos.
- **Ofrecer una interfaz intuitiva y elegante:** brindar una herramienta de uso sencillo para un usuario no técnico, con una interfaz gráfica agradable, **funcional y clara**, que no abrume con información pero que permita acceder rápidamente a los datos importantes.

Alcance del Proyecto

El sistema abarcará la **gestión interna** de la escuela de yoga de Minouche. **Incluye** funcionalidades de registro de alumnos, control de cuotas y pagos, gestión de gastos e ingresos, y emisión de informes y resúmenes. Se enfocará en las necesidades de *una única usuaria* (la administradora/profesora), en un contexto **offline** (sin necesidad de acceso a Internet).

Quedan **fuera del alcance** de esta versión inicial funciones más propias de sistemas complejos o en línea, como por ejemplo la reserva online de clases por parte de alumnos, integraciones con pagos en línea, manejo de múltiples usuarios con diferentes roles, o aplicaciones móviles. No obstante, en la

sección de mejoras futuras se mencionan algunas de estas características por si en el futuro se desea ampliar el sistema.

Requisitos Funcionales

A continuación, se detallan las funcionalidades **esenciales** que deberá ofrecer la aplicación:

Gestión de Alumnos

- **Ficha de alumno:** El sistema permitirá **crear, visualizar, modificar y eliminar** registros de alumnos. Cada ficha de alumno incluirá los **datos habituales** de identificación y contacto (por ejemplo: nombre completo, dirección, teléfono, correo electrónico) ¹, así como datos adicionales relevantes:
- **Información personal:** edad, fecha de nacimiento (opcional), y cualquier dato de salud o consideración especial que la profesora deba tener en cuenta (por ejemplo, lesiones o preferencias).
- **Datos de contacto:** teléfono, email y dirección postal. En caso de emergencia, se podría incluir un contacto alternativo (familiar, etc.).
- **Notas/comentarios:** campo de texto libre para agregar comentarios sobre el alumno (por ejemplo, su progreso, disponibilidad, observaciones particulares de comportamiento, etc.).
- **Estados del alumno:** Se podrá marcar el estado actual de cada alumno para saber si está **activo** (asistiendo actualmente), en **baja temporal** (ha pausado su asistencia por un tiempo), o **baja definitiva** (se dio de baja permanentemente) ². Esto ayudará a la gestión, permitiendo filtrar o distinguir alumnos que ya no asisten.
- **Historial:** Mantener un histórico simple de cambios importantes en la ficha (por ejemplo, cambio de plan de pagos, notas de cuando estuvo de baja, etc.) en caso de ser necesario para referencia.
- **Búsqueda y filtrado:** La aplicación ofrecerá una forma rápida de buscar alumnos por nombre u otros criterios, así como filtrar la lista (por ejemplo, ver solo alumnos activos, o solo los que tienen pagos pendientes, etc.).
- **Listado de alumnos:** Visualización en formato tabla de todos los alumnos registrados, mostrando campos clave (nombre, contacto, estado, y quizá su último pago realizado). Desde esta lista se debe poder seleccionar un alumno para ver/editar detalles.

Gestión de Pagos e Ingresos (Cuotas de los Alumnos)

- **Registro de cuotas:** Por cada alumno, el sistema permitirá **registrar los pagos de sus cuotas** (mensuales u otro periodo definido). Se debe poder indicar la fecha del pago, el periodo que cubre (ejemplo: mes de septiembre 2025) y el monto abonado si aplica. La aplicación debe marcar claramente si un alumno **está al corriente** en sus pagos o tiene cuotas pendientes por pagar, según los registros de meses:
- Por ejemplo, en la ficha del alumno o en el listado general se podrá ver un indicador de "Pago al día" o "Pago pendiente" de acuerdo con la última cuota registrada.
- También podría implementarse una sección donde se vea un calendario o lista de meses para cada alumno con una marca de pagado/no pagado.
- **Notificaciones internas:** (Opcional) Al ingresar al sistema, la pantalla principal podría resaltar alumnos con pagos atrasados (por ejemplo, lista de deudores) para que la profesora pueda hacer seguimiento.
- **Emisión de recibos/facturas:** Cada vez que un alumno pague su cuota, la aplicación dará opción de **generar un recibo o factura** por ese pago. El documento deberá incluir los datos del centro (nombre de la escuela, datos fiscales si fueran necesarios), los datos del alumno, la fecha

y concepto del pago (ej. "*Cuota mensual - Septiembre 2025*"), y el importe. Estos comprobantes se podrán:

- **Imprimir o guardar en PDF** para entregar al alumno o archivo de la escuela.
- **Guardar** en el sistema para consulta futura (es decir, mantener un historial de pagos con su recibo asociado).
- **Consulta de historial de pagos:** La profesora podrá consultar en cualquier momento los pagos realizados por un alumno específico (historial de pagos del alumno) o bien **listados globales** de pagos:
 - Por ejemplo, un listado de todos los ingresos recibidos en un mes determinado, o entre dos fechas (útil para contabilidad).
 - Sumatoria total de ingresos por año o por mes, etc., para ver tendencias.
- **Diferentes métodos de pago:** (Opcional, según necesidad) Permitir registrar el **método de pago** utilizado por el alumno: efectivo, transferencia, tarjeta, etc., en caso de querer llevar ese detalle. Esto podría ser útil para reportes (saber cuántos pagos fueron en efectivo vs transferencia, por ejemplo).
- **Tarifas o planes:** En principio todos los alumnos podrían manejar el mismo esquema de cuota mensual. Sin embargo, el sistema será flexible para soportar **diferentes tarifas** si fuera necesario:
 - Por ejemplo, se podría configurar que algunos alumnos pagan mensual, otros trimestral, o que hay distintos niveles de precio (tarifa normal, tarifa reducida para desempleados, etc.). Cada alumno tendría asociado qué tipo de tarifa aplica.
 - Si hubiera **bonos de clases** (p. ej., paquetes de 10 clases en vez de mensualidad), el sistema también debería poder manejarlos registrando la compra del bono y descontando clases a medida que asiste, aunque esto ya representa lógica adicional no solicitada explícitamente. Se menciona aquí como posible extensión.
- **Cobros y seguimiento:** La aplicación facilitará realizar los cobros puntuales y registrar su pago en el sistema. Una vez marcado un pago como realizado, se refleja en el estado del alumno y se actualiza la información contable. *(En caso de integrar con banca o plataformas de pago en el futuro, se podría semi-automatizar esta función, pero inicialmente será un registro manual hecho por la profesora.)*

Gestión de Gastos (Contabilidad de Egresos)

- **Registro de gastos:** La aplicación permitirá **registrar todos los gastos** relacionados con la escuela de yoga. Para cada gasto se almacenará:
 - **Descripción o concepto** del gasto (ejemplo: "*Factura electricidad - Septiembre*", "*Compra de esterillas de yoga*", "*Publicidad en redes sociales*", etc.).
 - **Fecha** del gasto (cuándo se pagó).
 - **Importe** pagado.
 - **Categoría** del gasto: al menos distinguir si es un **gasto fijo** o **gasto variable**. Los **gastos fijos** son aquellos recurrentes y necesarios (por ejemplo, alquiler del local, facturas de suministros como luz, agua, etc.) ³, mientras que los **gastos variables** son más flexibles o eventuales (por ejemplo, material de limpieza, reemplazo de colchonetas, gastos en marketing o eventos especiales).
 - (Opcional) **Proveedor o destinatario** del pago, si se desea llevar registro de a quién se le paga (por ejemplo, compañía eléctrica XYZ, tienda de yoga ABC).
 - (Opcional) **Documento adjunto:** posibilidad de adjuntar un archivo (imagen/PDF) de la factura o recibo del gasto para tener respaldo digital, aunque esto puede omitirse en aras de la simplicidad y dado que incrementaría los datos almacenados.
- **Gastos fijos vs variables:** En el sistema se podrá **clasificar** cada gasto como fijo o variable. Además:

- Se podrían pre-configurar ciertos gastos fijos típicos que ocurren cada mes (por ejemplo, "Alquiler local", "Luz", "Agua") para agilizar su registro periódico. Estos gastos fijos suelen ser periódicos y de importes relativamente constantes o previsibles ³.
- Los gastos variables podrían categorizarse adicionalmente por tipo (por ejemplo, "Materiales", "Mantenimiento", "Marketing", "Impuestos/ tasas", etc.) para facilitar reportes más detallados, aunque inicialmente con marcarlos como variables es suficiente.
- **Resumen de gastos:** El sistema debe ofrecer vistas o informes que muestren **el total de gastos** en un período, desglosando entre fijos y variables. Ejemplos:
 - Total gastado en el mes actual, y cuánto de eso fue en gastos fijos (obligatorios) vs variables.
 - Gastos fijos mensuales previstos (una lista de los gastos fijos típicos y su monto, para saber los compromisos mensuales).
 - Tendencia de gastos a lo largo de varios meses (si se desea, se podría graficar o listar para ver si ciertos gastos aumentan o disminuyen).
- **Comparativo con ingresos:** Sería útil que, junto con el resumen de gastos, se pueda ver **la comparación con los ingresos** del mismo período para tener una idea del balance neto (beneficio o pérdida). Por ejemplo, un informe mensual podría mostrar: "Ingresos totales vs Gastos totales" de ese mes, y la diferencia.
- **Gestión de facturas de gastos:** Si la profesora recibe facturas por servicios (por ejemplo la factura de la luz, agua, alquiler), éstas pueden ser numeradas y registradas. El sistema no necesita llevar contabilidad fiscal completa, pero **almacenará los datos básicos**:
 - Número de factura (opcional, si se quiere llevar control de facturas proveedores),
 - Fecha de emisión y pago,
 - Monto,
 - Proveedor.
 - Esto facilita luego encontrar un gasto específico si se requiere (p.ej., buscar "factura de luz de agosto").

Informes y Exportaciones de Datos

- **Panel de resumen (dashboard):** La aplicación ofrecerá una pantalla o sección de **resumen general** con indicadores clave, para que la profesora pueda ver de un vistazo la situación de la escuela. En este panel se puede incluir, por ejemplo:
 - Número total de alumnos activos (y opcionalmente cuántos nuevos se inscribieron en el último mes, cuántos causaron baja).
 - Cantidad de alumnos con pago al día vs alumnos con pagos pendientes.
 - Ingresos totales del mes en curso vs gastos totales del mes en curso.
 - Un pequeño gráfico o listado de ingresos y gastos en meses recientes (evolución).
 - Cualquier alerta importante (por ejemplo "El seguro anual vence pronto", si se registró algo de ese estilo como gasto próximo, o "Alumno X tiene 2 meses sin pagar").
- **Generación de informes:** Se podrán generar **informes detallados** en formatos comunes (Excel y PDF) con la información de la gestión. Algunos informes previstos:
 - **Listado de alumnos** con sus datos de contacto y estado, para imprimir o revisar fuera de la aplicación.
 - **Informe de pagos:** por ejemplo, un informe mensual que liste todos los pagos de cuotas recibidos ese mes, sumando el total, y detallando qué alumnos no pagaron.
 - **Informe de ingresos y gastos:** un balance simple en PDF o Excel que muestre todos los ingresos y gastos en un periodo seleccionado (ej. un año, o un trimestre), con subtotales por categoría. Esto ayuda a la contabilidad y a preparar declaraciones o simplemente a entender la rentabilidad.

- **Gráficos:** aprovechando la exportación a Excel, se pueden generar gráficos (o datos listos para graficar) sobre la evolución de alumnos o finanzas. Por ejemplo, número de alumnos activos por mes a lo largo del año, o ingresos vs gastos por mes.
- **Exportación a Excel:** Dado que la usuaria está familiarizada con hojas de cálculo, la aplicación podrá **exportar datos a un archivo Excel (.xlsx)** de forma que se pueda abrir y examinar externamente. Esto puede incluir:
 - Exportar la lista de alumnos completa a Excel (como respaldo o para análisis adicional personalizado).
 - Exportar los registros de pagos o gastos.
 - Exportar el resumen financiero (quizá en una hoja con varias tablas o pivotes).
- La integración con Excel es importante ya que permite tener **flexibilidad** en la creación de informes personalizados ⁴ ; de esta manera, la profesora puede realizar cálculos o gráficos adicionales por su cuenta si lo desea.
- **Exportación/guardado en PDF:** Además de los recibos ya mencionados en la sección de pagos, cualquier informe generado debería ofrecer la opción de guardarse/emirse en formato PDF. Esto es útil para archivarlos digitalmente o imprimirlos.
- **Impresión:** La aplicación debe soportar la impresión directa de los informes y recibos importantes, formateados adecuadamente para papel tamaño estándar (A4, por ejemplo).
- **Copias de seguridad de datos:** Si bien no es exactamente un informe, es relevante mencionar que la aplicación debería permitir **guardar/respaldar toda la información** en un formato accesible (por ejemplo, exportar todo a Excel o a un archivo de respaldo) periódicamente, para protección de datos. En caso de fallo del sistema o cambio de equipo, poder recuperar la información desde ese backup.

Interfaz de Usuario (UI)

- **Sencillez y funcionalidad:** La interfaz gráfica será **sencilla, intuitiva y elegante** (agradable a la vista). Se evitarán pantallas recargadas; la información se organizará de forma clara con menús o pestañas para cada módulo principal (Alumnos, Pagos/Ingresos, Gastos, Informes).
- **Navegación clara:** Desde una pantalla de inicio (o menú principal) se podrá acceder a cada funcionalidad fácilmente. Por ejemplo, un menú lateral o superior con botones: "**Alumnos**", "**Pagos**", "**Gastos**", "**Informes**", y "**Configuración**". La usuaria debe entender rápidamente dónde hacer clic para cada tarea.
- **Diseño consistente:** Todas las pantallas compartirán un diseño coherente (mismos colores, fuentes legibles, botones en posiciones consistentes). Posiblemente se utilicen colores neutros o relacionados con tranquilidad (dado el contexto de yoga, podría emplearse una paleta suave).
- **Formulario y tablas:** La gestión de datos (como altas de alumnos o registro de gastos) se hará mediante **formularios simples** con campos claramente etiquetados. Las listas de datos (como listado de alumnos o de gastos) se mostrarán en **tablas** con posibilidad de ordenar y scroll, asegurando que incluso en un portátil de baja resolución se pueda ver la información sin complicaciones (posiblemente adaptando el tamaño de ventana o usando barras de desplazamiento).
- **Validación y errores:** La interfaz validará entradas para evitar errores comunes. Por ejemplo:
 - Si se deja un campo obligatorio vacío (como nombre del alumno) al guardar, mostrará un mensaje indicando completar la información.
 - Controlará formatos básicos (p.ej., si se ingresa texto en un campo numérico de importe).
 - Prevendrá acciones destructivas accidentales: si se va a eliminar un alumno o borrar un registro, pedirá confirmación.
- **Elegancia:** Además de la funcionalidad, se busca un toque **estético profesional**. Esto implica alineación ordenada, quizás incluir el logotipo de la escuela (si lo tiene) en la ventana principal o

en reportes, íconos representativos en botones (por ejemplo, un ícono de persona para Alumnos, un ícono de moneda o ticket para Pagos, un gráfico para Informes).

- **Ayuda y feedback:** La UI ofrecerá feedback inmediato de las acciones del usuario. Por ejemplo, tras guardar un registro exitosamente, mostrar un pequeño mensaje tipo "Alumno guardado correctamente". Si ocurre un error, indicar qué pasó. Asimismo, se puede incluir un apartado de "Ayuda" o pequeños textos informativos en cada pantalla para guiar (por ejemplo, en la pantalla de Informes un texto "Seleccione un mes y pulse 'Generar informe' para ver el balance").
- **Idioma:** La interfaz estará en **español** (castellano), acorde al usuario. Todos los campos y mensajes deben estar en español claro. (En un futuro, si se requiriera, se podría pensar en internacionalización, pero no es necesario ahora).
- **No sobrecargar recursos:** Dado que correrá en un portátil modesto, la interfaz debe ser ligera: evitar animaciones pesadas, transiciones complejas o elementos que consuman mucha memoria. Se privilegiará el uso de controles estándar y ventanas simples.

Funciones Adicionales y Mejoras Futuras

(Estas características no son requisitos obligatorios para la primera versión, pero se mencionan como ideas para posibles mejoras o expansiones del sistema, en caso de que la escuela crezca o surjan nuevas necesidades.)

- **Control de asistencia a las clases:** Implementar un módulo donde la profesora pueda marcar qué alumnos asistieron a cada clase o sesión. Esto permitiría llevar un **registro histórico de asistencias**, y podría ser útil tanto para la profesora (ver qué tan constante es cada alumno) como para ciertos modelos de tarifa (por ejemplo, si algún alumno paga por clase o tiene un bono limitado). Un buen sistema de gestión de un estudio suele incluir el registro de asistencia y seguimiento de clases impartidas ⁵. En nuestro caso de uso actual (una sola profesora con sus alumnos), esto podría ser tan simple como una lista de alumnos por día donde se tildan los presentes.
- **Programación de clases:** Si la escuela ofrece múltiples horarios o tipos de clase, se podría incorporar un calendario o agenda donde planificar las clases y quizá **asociar alumnos o grupos** a esas clases. Por ejemplo, crear horarios (Lunes 18:00 Yoga Nivel I, Martes 10:00 Yoga Suave, etc.) y vincular la asistencia de alumnos. Actualmente Minouche probablemente ya maneja un horario fijo, pero este módulo sería útil si en el futuro contrata más profesores o amplía la oferta.
- **Gestión de grupos o niveles:** Relacionado con lo anterior, poder **agrupar alumnos** por categorías (por ejemplo, grupo de mañana vs grupo de tarde, o nivel principiante vs avanzado). Esto facilita enviar comunicaciones o gestionar pagos por grupo si las tarifas difieren. La aplicación GesYoga, por ejemplo, permite crear grupos de práctica y asignar alumnos a ellos ⁶, lo cual puede ser una referencia para futuras expansiones.
- **Envío de recordatorios/notificaciones:** Integrar funcionalidades para enviar recordatorios a los alumnos, por ejemplo:
 - Un aviso por correo electrónico o SMS a quienes no hayan pagado su cuota cuando corresponde.
 - Recordatorios de eventos especiales (talleres, cambios de horario) a todos los alumnos. *Esta funcionalidad requeriría conexión a Internet y posiblemente una integración con un servicio de correo o mensajería, por lo que se consideraría más adelante.*
- **Soporte multi-usuario:** Si en algún momento hubiera más personal gestionando la escuela (por ejemplo, otra persona que ayuda con la administración, o si la escuela crece con varios profesores que requieran acceso), podría implementarse un sistema de usuarios con credenciales y distintos niveles de permiso (administrador vs solo lectura, etc.). Actualmente, al

ser solo la profesora, **no se implementará gestión de usuarios** ni contraseñas de acceso más allá quizás de la propia seguridad de Windows.

- **Módulo de inventario:** En caso de que la escuela maneje inventario (ej: mats, bloques, cojines para venta o alquiler, etc.), se podría añadir un seguimiento de inventario y ventas de productos. Esto de nuevo sería una ampliación futura si Minouche decide vender artículos de yoga u ofrecer algo extra.
- **Integración con contabilidad externa:** Exportar los datos de ingresos/gastos a algún formato contable estándar (CSV, Excel estructurado) que pudiera ser utilizado en software de contabilidad o entregarse a un gestor contable. Ya mencionamos la exportación a Excel/PDF, lo cual cubre gran parte de esta necesidad.
- **Aplicación móvil o acceso remoto:** A largo plazo, si se necesitara consultar datos desde fuera del portátil (por ejemplo, la profesora quiere ver la lista de alumnos o confirmar pagos desde su teléfono), se podría desarrollar una versión móvil o una interfaz web sincronizada. Esto implicaría una arquitectura cliente-servidor o en la nube, claramente fuera del alcance de la versión actual, pero es una mejora posible si la digitalización del negocio avanza.

Requisitos Técnicos y de Implementación

- **Plataforma:** La aplicación se ejecutará en **Windows** (el portátil de la profesora usa Windows como sistema operativo). Se deberá asegurar la compatibilidad al menos con versiones Windows 10 y superiores. No se descarta que pueda funcionar en Windows 7 u 8 si fuera necesario, pero idealmente se desarrollará para entornos soportados actualmente (Windows 10/11).
- **Rendimiento y uso de recursos:** Debe ser un programa **ligero**, capaz de correr en un portátil con especificaciones modestas (por ejemplo, 4 GB de RAM, procesador básico). La aplicación no realizará operaciones intensivas, por lo que se espera que su consumo de CPU y memoria sea bajo. Se optimizará para que abra rápido y responda con agilidad a las interacciones de la usuaria, incluso con ~hundreds~ decenas de registros en las listas.
- **Almacenamiento de datos:** Dado el **bajo volumen de datos** esperado (la escuela tiene un número manejable de alumnos y registros de pagos/gastos), no es necesario un servidor de base de datos grande. Se plantean dos enfoques posibles:
 - Utilizar una **base de datos embebida ligera**, como SQLite, la cual almacena la información en un solo fichero en el disco y **no requiere instalación de un motor de base de datos** por separado. SQLite es muy adecuada para aplicaciones de escritorio pequeñas.
 - Alternativamente, utilizar **archivos de texto estructurados** (por ejemplo, JSON, CSV o XML) para guardar la información. Por ejemplo, un archivo para alumnos, otro para pagos, etc. Esta opción simplifica la implementación pero requiere mayor cuidado manual para asegurar la integridad de los datos. Solo se recomendaría si la estructura de datos es muy simple. En todo caso, la aplicación manejaría la lectura/escritura de dichos archivos de forma transparente al usuario.
- **Respaldo de información:** Independientemente del método, el almacenamiento local implica riesgo si el equipo se daña. Se recomienda que la aplicación incluya una opción sencilla de **“Exportar/Backup de datos”**, que comprima todos los datos (o los vuelque a un Excel global) para que la profesora pueda guardarlo en un USB u otro PC como seguridad.
- **Lenguaje de programación:** Se deberá elegir un lenguaje/tecnología que cumpla con el requerimiento de **facilidad de implementación y mínimos requisitos de instalación**. Esto significa:
 - Idealmente, la aplicación se distribuirá como un **ejecutable único** o un instalador muy simple, evitando obligar al usuario a instalar runtimes o dependencias pesadas.
 - Lenguajes adecuados podrían ser:
 - **Python** (por su rapidez de desarrollo y amplia disponibilidad de librerías) creando la interfaz gráfica con Tkinter, PyQt o similar, y utilizando herramientas como PyInstaller

para generar un `.exe` autónomo que incluya el intérprete. Esto permitiría no tener que instalar Python por separado.

- **Java o Kotlin** con JavaFX/Swing podría considerarse, generando un runnable jar, pero requeriría tener la JVM; se podría incluir un JRE portátil en el paquete.
- **C# / .NET**: Crear una aplicación de escritorio (WinForms/WPF) que funcione con .NET Framework (ya incluido en Windows en muchas versiones) o .NET 6/7 autónomo. Esto daría un ejecutable y posiblemente unos archivos de runtime, pero Microsoft ofrece opciones de *self-contained deployment* (empaquetar el runtime).
- **C++** con Qt u otra librería de GUI nativa para obtener un ejecutable muy eficiente; sin embargo, el desarrollo podría ser más complejo que usando lenguajes de más alto nivel.
- Incluso se podría considerar una **solución basada en Excel (macros/VBA)** dado que Excel ya está presente en muchos entornos Windows. De hecho, existen aplicaciones como *GesYoga* que operan enteramente sobre Excel ⁴. Sin embargo, esto puede ser menos robusto y más difícil de mantener en el tiempo que una aplicación independiente.
- En resumen, se priorizará el lenguaje que permita **menor fricción al instalar**. Por ejemplo, una pequeña aplicación Python compilada a exe cumple esto, o un ejecutable .NET si el sistema ya tiene las librerías necesarias. El detalle de la selección se terminará de definir en base a la experiencia del desarrollador y las pruebas en el hardware destino.
- **Dependencias de terceros**: Se buscará minimizar el uso de librerías o componentes externos que el usuario deba instalar manualmente. Si se usan librerías (por ejemplo, una librería para generar PDF o Excel), estas se incluirán en el paquete de la aplicación.
- **Seguridad de datos**: Dado que es una aplicación local monousuario, no se implementará un sistema complejo de seguridad o cifrado. Sin embargo, por privacidad, los datos de alumnos deben almacenarse de forma segura en el sentido de evitar acceso no deseado:
- Si el equipo es compartido o no está protegido, opcionalmente se podría **proteger el acceso a la aplicación con una contraseña** maestra (para que solo la profesora entre). Esto es opcional considerando que Minouche probablemente usa su propio portátil.
- Los datos podrían cifrarse ligeramente en los archivos de almacenamiento para que no sean legibles a simple vista, aunque esto no es crítico.
- **Mantenibilidad y extensibilidad**: El código de la aplicación se escribirá siguiendo buenas prácticas para facilitar futuras modificaciones. Esto incluye documentación interna (comentarios), estructura modular (separando lógica de negocio de la interfaz gráfica, por ejemplo) y posibilitar agregar módulos nuevos (como los mencionados en mejoras) sin tener que reescribir desde cero.
- **Pruebas y calidad**: Antes de la entrega, se probará la aplicación con escenarios típicos:
- Alta de varios alumnos, registro de pagos, simulación de meses sucesivos para ver cómo se reflejan los pendientes.
- Ingreso de varios gastos, marcándolos fijos y variables, y generación de un informe de balance.
- Exportación a Excel y apertura de esos archivos en Excel para verificar la correcta estructura de los datos.
- Verificación de que la aplicación sigue funcionando fluidamente con, por ejemplo, 100 alumnos y un año de registros (para asegurarse de que escalas pequeñas no la ralenticen).
- **Limitaciones conocidas**: Se consignará que la aplicación, al ser local, no previene pérdida de datos por daños físicos al equipo, por lo que se recomendará al usuario hacer backups periódicos (como mencionado). También, no habiendo funcionalidad en la nube, la accesibilidad es solo desde el portátil donde esté instalada.

Conclusión

En este documento se han detallado **todos los requisitos y especificaciones** para la aplicación de gestión de la escuela de yoga de Minouche. El enfoque principal es una herramienta **simple pero**

completa que cubra la **gestión de alumnos y las finanzas** del centro, con énfasis en facilidad de uso y bajo requerimiento técnico.

La siguiente etapa, una vez validados estos requisitos con la profesora (usuario final), será proceder al diseño de la solución técnica y la implementación. Con esta especificación exhaustiva, se busca que el desarrollo (sea manual o asistido por herramientas de generación de código) tenga una guía clara de qué construir. El resultado esperado es una aplicación que facilite la administración diaria de la escuela de yoga, permitiendo a la profesora **centrarse más en la enseñanza** y menos en las tareas administrativas, con la tranquilidad de llevar sus registros al día y de forma ordenada.

-
- 1 2 4 6 GesYoga, aplicación gratuita para la gestión de tu centro de yoga | Yoga en Red
<https://www.yogaenred.com/2016/02/04/gesyoga-aplicacion-gratuita-para-la-gestion-de-tu-centro-de-yoga/>
 - 3 Gastos fijos y variables, ¿en qué consisten y cómo se diferencian?
<https://www.bbva.com/es/salud-financiera/gastos-fijos-y-variables-en-que-consisten-y-como-se-diferencian/>
 - 5 Guía Definitiva: Todo lo que debes tener en cuenta para abrir tu Escuela de Yoga | Tribu Yoga
<https://www.tribu.yoga/blog/guia-definitiva-todo-lo-que-debes-tener-en-cuenta-para-abrir-tu-escuela-de-yoga/>